Artificial Neural Network

CEP REPORT



Submitted By

23BSAI048,23BSAI001,23BSAI006

1. Introduction

This report presents a deep learning project for **automatic pavement and concrete crack detection**. Given a photo of a road or concrete surface, the system decides whether it contains a crack (**Positive**) or not (**Negative**). Two neural networks were trained from scratch and compared: a **Convolutional Neural Network (CNN)** and a **Vision Transformer (ViT)**. The better model, the CNN with **98.27% test accuracy**, was exported and deployed in an interactive **Streamlit web application**.

The complete pipeline (data loading, preprocessing, exploratory data analysis, feature engineering, model building, training, evaluation and model export) is implemented in one Google Colab notebook: pavement_crack_detection.ipynb

Deploy Project Link: Pavement Crack Detector · Streamlit

2. Problem Statement

Roads, bridges and buildings develop cracks over time because of traffic loads, weather and ageing. Manual visual inspection of large surfaces is slow, costly, subjective and sometimes dangerous. This project automates the first step of inspection with deep learning. Its objectives are to:

- **Detect** whether a surface image contains a crack (binary classification).
- **Compare** a CNN with a Vision Transformer trained from scratch on the same data and pipeline.
- **Deploy** the best model in a simple web app that accepts any image format, a live camera photo, or built-in demo images.

3. Dataset Description

The project uses the Kaggle dataset **Concrete and Pavement Crack Images** (oluwaseunad/concrete-and-pavement-crack-images), downloaded inside the notebook with **kagglehub**. It contains **30,000 RGB .jpg images** in two perfectly balanced classes (15,000 Negative and 15,000 Positive). The images are small surface patches, mostly 227×227 or 227×170 pixels. The data was split with stratification (random seed 42) into training, validation and test sets. The test set was not used until the final evaluation.

Split	Negative	Positive	Total	Share
Train	10,500	10,500	21,000	70%
Validation	2,250	2,250	4,500	15%

Split	Negative	Positive	Total	Share
Test	2,250	2,250	4,500	15%
Total	**15,000**	**15,000**	**30,000**	**100%**

Table 1: Stratified dataset split

3.1 Exploratory Data Analysis

The two classes are perfectly balanced, so the balanced class weights are 1.0 for both classes and no resampling is needed.



Figure 1: *Class distribution (count and share)*

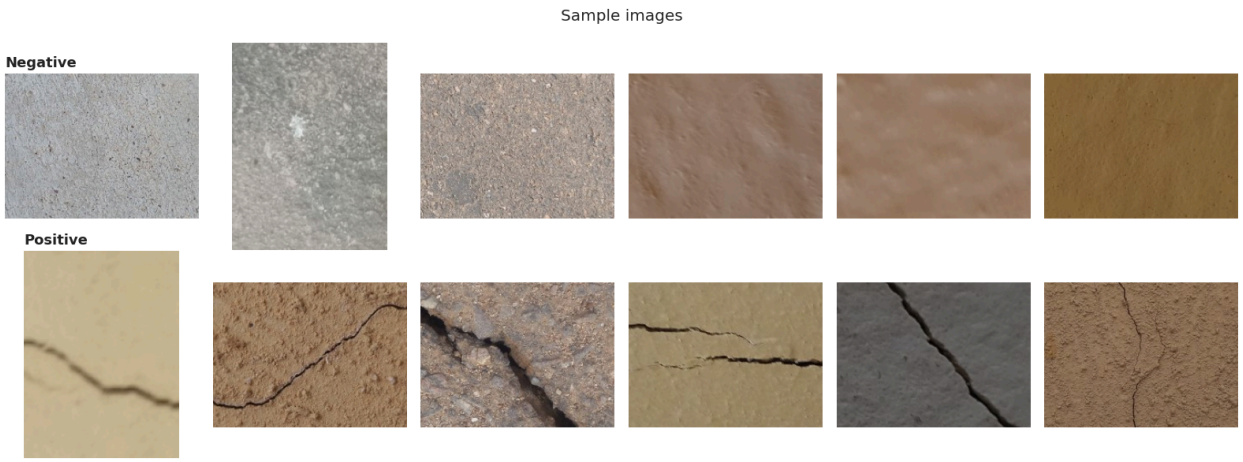


Figure 2: *Random sample images. Top: Negative (no crack). Bottom: Positive (crack).*

On a sample of 300 images per class, average **brightness** is almost the same (132.4 vs 131.3), but **contrast** (pixel standard deviation) is clearly higher for cracked images (18.2 vs 12.7), because cracks are thin dark lines on a lighter background. Contrast and edges carry the useful signal, not overall brightness.

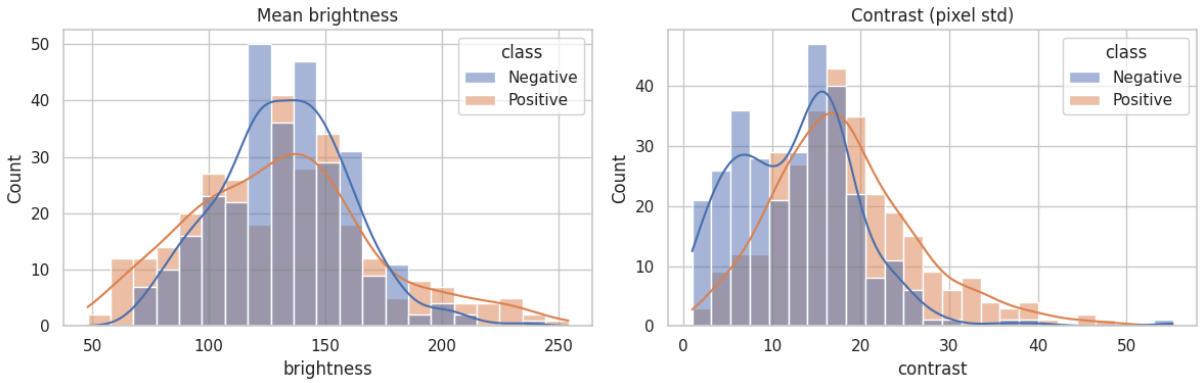


Figure 3: *Brightness and contrast distribution per class*

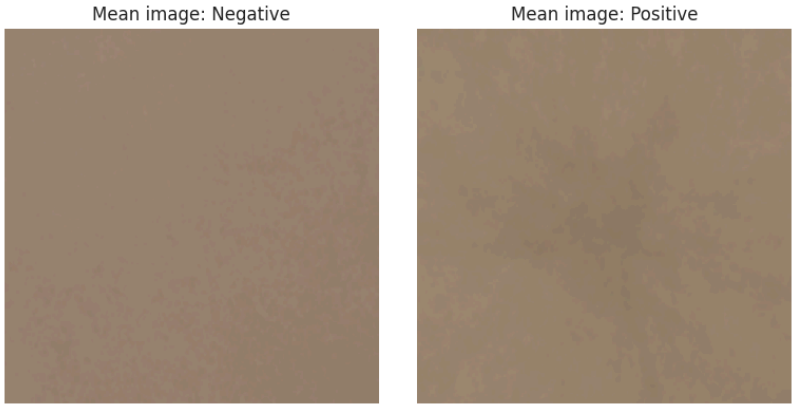


Figure 4: *Mean image per class: crack positions are random, so they average out*

4. Methodology

4.1 Data Preprocessing

- **Integrity check:** every file was opened and verified with PIL, and duplicate paths were removed. All 30,000 images were valid.
- **Stratified split** 70 / 15 / 15 into train, validation and test (Table 1).
- **Class weights** computed with scikit-learn (balanced): 1.0 / 1.0.
- **tf.data pipeline:** read, decode, resize to **224×224**, convert to uint8, cache in memory, shuffle, batch (64), and prefetch. Images are loaded lazily from file paths to save RAM.
- **Normalization** (rescaling to 0 to 1) is a layer **inside** each model, so the deployed app passes raw RGB pixels and uses exactly the same preprocessing as training.

4.2 Feature Engineering

Edge analysis: Sobel edge maps show what the networks must learn. Cracks produce strong, continuous edges, while intact surfaces only produce weak, scattered texture edges.

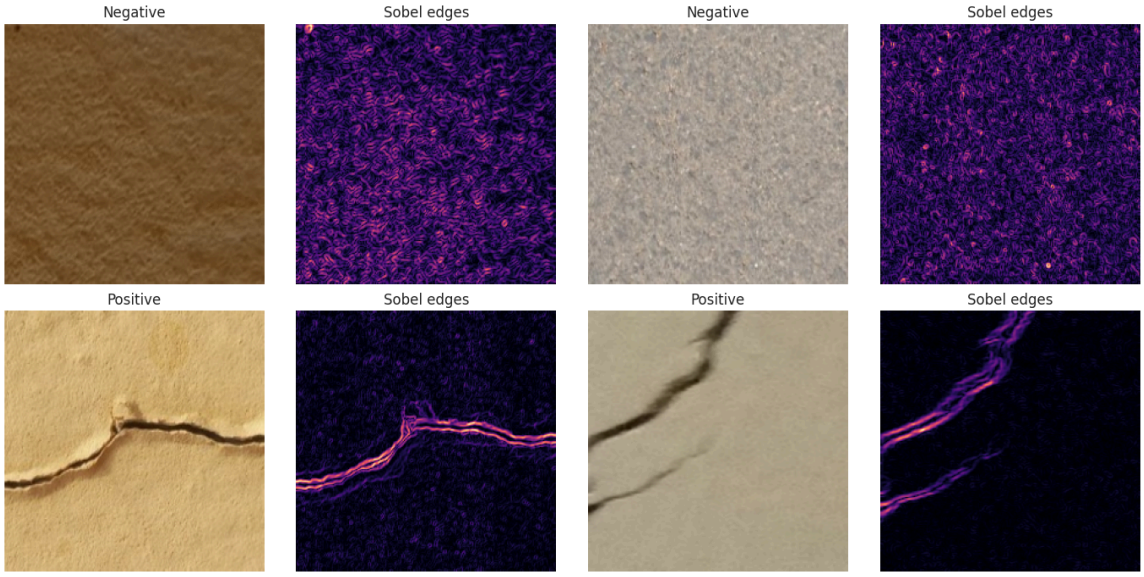


Figure 5: *Original images and their Sobel edge maps*

Data augmentation (training set only) creates new variations of each image every epoch, which reduces overfitting: random horizontal and vertical flips, rotation (±36°), zoom (±10%), contrast (±15%) and brightness (±10%).



Figure 6: *Eight augmented views generated from one training image*

4.3 Model 1: Convolutional Neural Network (CNN)

- Input 224×224×3, followed by a **Rescaling** (1/255) layer.
- **4 convolution blocks** with 32, 64, 128 and 256 filters. Each block is Conv2D 3×3, then BatchNormalization, ReLU and MaxPooling 2×2.
- **GlobalAveragePooling2D**, **Dropout 0.4**, and **Dense(1, sigmoid)**, which outputs P(crack).
- Total parameters: **390,113**.

4.4 Model 2: Vision Transformer (ViT)

The ViT treats an image as a sequence of patches, the same way a Transformer treats a sentence as a sequence of words. Each 224×224 image is split into **196 patches of 16×16 pixels** (768 values each).

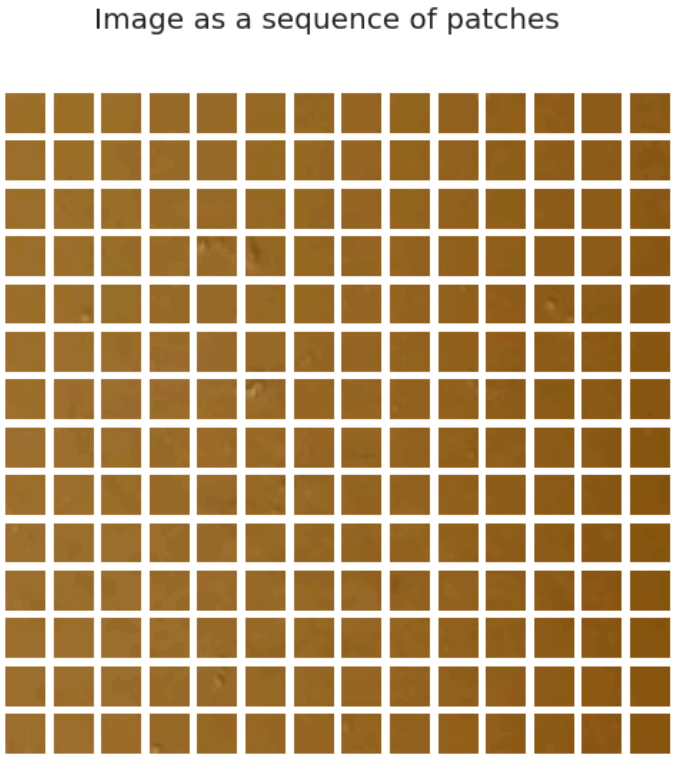


Figure 7: *One image split into a 14×14 grid of 16×16 patches*

- **Patch embedding:** Conv2D with 64 filters, kernel 16 and stride 16 (a linear projection of each patch to 64 dimensions).
- **Learnable position embedding** (custom PositionEmbedding layer), so the model knows where each patch came from.
- **6 Transformer encoder blocks.** Each one applies LayerNorm, Multi-Head Self-Attention (4 heads) and a residual connection, then LayerNorm, an MLP (128 units, GELU, dropout 0.1) and another residual connection.
- **LayerNorm**, **GlobalAveragePooling1D**, **Dropout 0.3**, and **Dense(1, sigmoid)**. Total parameters: **262,785**.

4.5 Training Setup

Setting	Value
Optimizer	AdamW (learning rate 1e-3, weight decay 1e-4)
Loss function	Binary cross-entropy

Setting	Value
Batch size / epochs	64 / 5
Callbacks	EarlyStopping (val_loss, restore best weights), ReduceLROnPlateau (×0.3, patience 2), ModelCheckpoint
Class weights	Balanced (1.0 / 1.0)
Metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC
Hardware / framework	Google Colab T4 GPU, TensorFlow 2.21.0, Keras 3.13.2
CNN training time	20.2 minutes (about 227 s per epoch)

Table 2: Training configuration (identical for both models)

5. Results and Discussion

5.1 Test-Set Performance

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	Parameters
CNN	**0.9827**	**0.9840**	**0.9813**	**0.9826**	**0.9977**	390,113
ViT	0.9253	0.9382	0.9107	0.9242	0.9833	262,785

Table 3: Results on the 4,500 unseen test images

The **CNN wins on every metric**. It misclassified only **78 of 4,500** test images (1.7%), compared with **336** (7.5%) for the ViT. Its ROC-AUC of 0.9977 means it separates the two classes almost perfectly across all thresholds.

5.2 Learning Curves

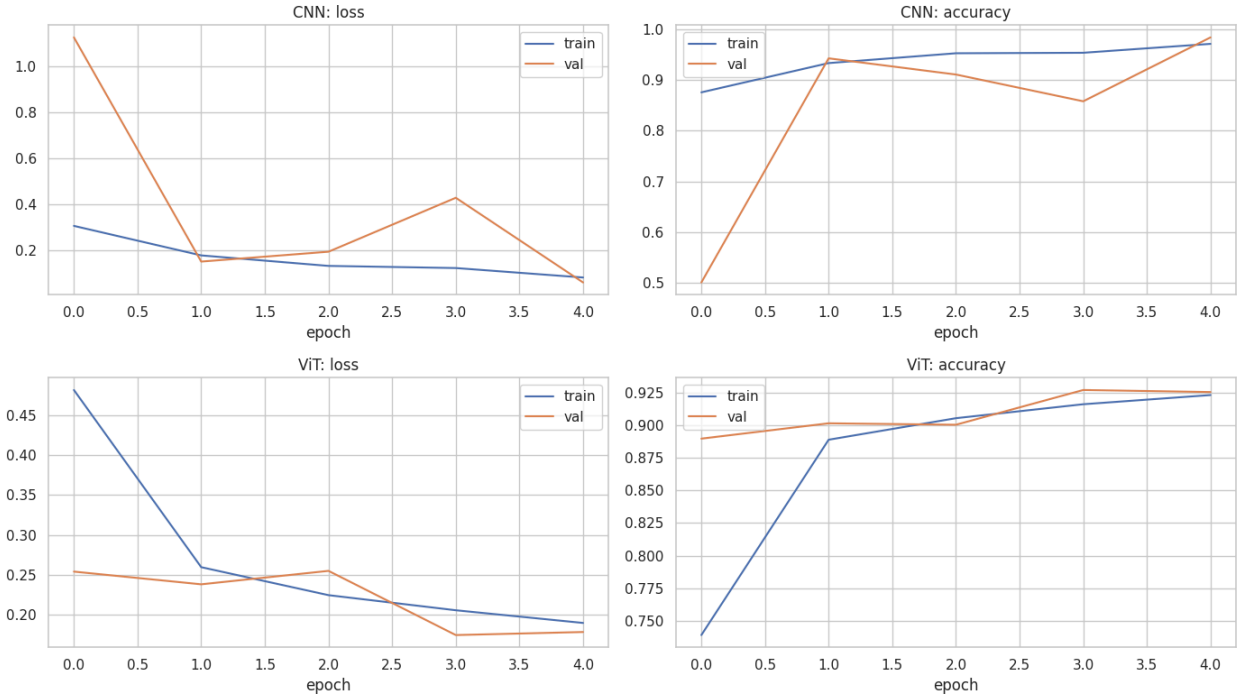


Figure 8: *Training and validation loss and accuracy per epoch*

The **CNN's** validation curve was unstable in epochs 2 to 4, with validation accuracy dropping to 0.86. This is typical when BatchNormalization statistics adjust at a high learning rate. ReduceLROnPlateau lowered the learning rate to 3e-4, and the final epoch reached **0.9844 validation accuracy** (validation loss 0.058). That epoch's weights were restored as the best model. The **ViT** trained smoothly, and its training and validation curves stay close (no overfitting). It was still improving at epoch 5, which shows it is under-trained.

5.3 Confusion Matrices

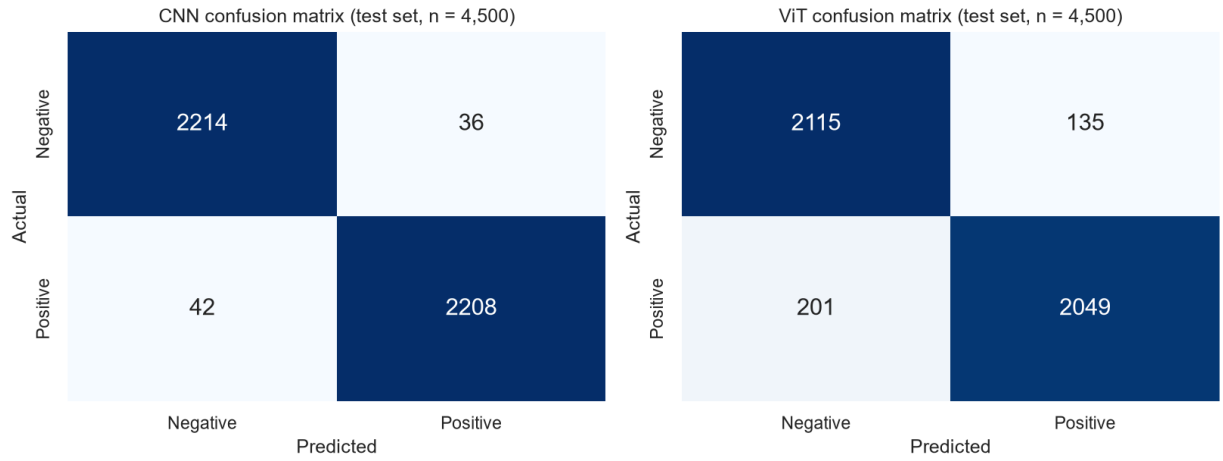


Figure 9: *Confusion matrices on the test set*

The CNN's errors are balanced: 36 false alarms and 42 missed cracks. The ViT missed **201 cracks**. In infrastructure inspection, a missed crack (false negative) is the more dangerous error, which makes the CNN clearly preferable.

5.4 Error Analysis



Figure 10: *Examples misclassified by the CNN, with the true label and predicted P(crack)*

Most remaining errors are **very thin, faint hairline cracks** on smooth coloured surfaces, which get low crack probability (0.21 to 0.35). The others are **intact surfaces with stains, rough texture or straight lines** such as joints and scratches, which look crack-like (one scored 0.99). Several of these are borderline, so the adjustable threshold in the app helps.

5.5 Why the CNN Outperformed the ViT

- **Inductive bias:** convolutions are built for local patterns and position-independent features, which suits thin local crack lines. A ViT has to learn these properties from data.
- **Data and pre-training:** published ViT results depend on pre-training on very large datasets (e.g. JFT-300M). With 21,000 training images and no pre-training, a ViT is at a disadvantage.

- **Short training:** both models trained for only 5 epochs, and the ViT's curves were still rising.

The ViT could be improved by training for more epochs, fine-tuning a pre-trained ViT, or using a hybrid CNN-Transformer.

6. Deployment: Streamlit Web Application

The best model (CNN) was saved as **crack_model.keras** together with **model_info.json** (class names, input size, threshold, test metrics, confusion matrix and library versions). Before download, the notebook reloads the saved model and checks that it gives the same predictions. The web app is built with **Streamlit** and provides:

- **Demo gallery:** 15 sample images with known labels. Users tick any of them and see the prediction next to the correct answer.
- **Upload any image format:** JPG, PNG, WEBP, HEIC (iPhone), TIFF, BMP, GIF, AVIF, plus grayscale, CMYK, 16-bit and transparent images. Every image is converted to 8-bit RGB and resized to 224×224. Files that are not images are skipped with a warning.
- **Camera input**, an adjustable **crack threshold**, result cards with confidence, and a **CSV download** of all predictions.
- A **Model performance** tab comparing the CNN and the ViT.

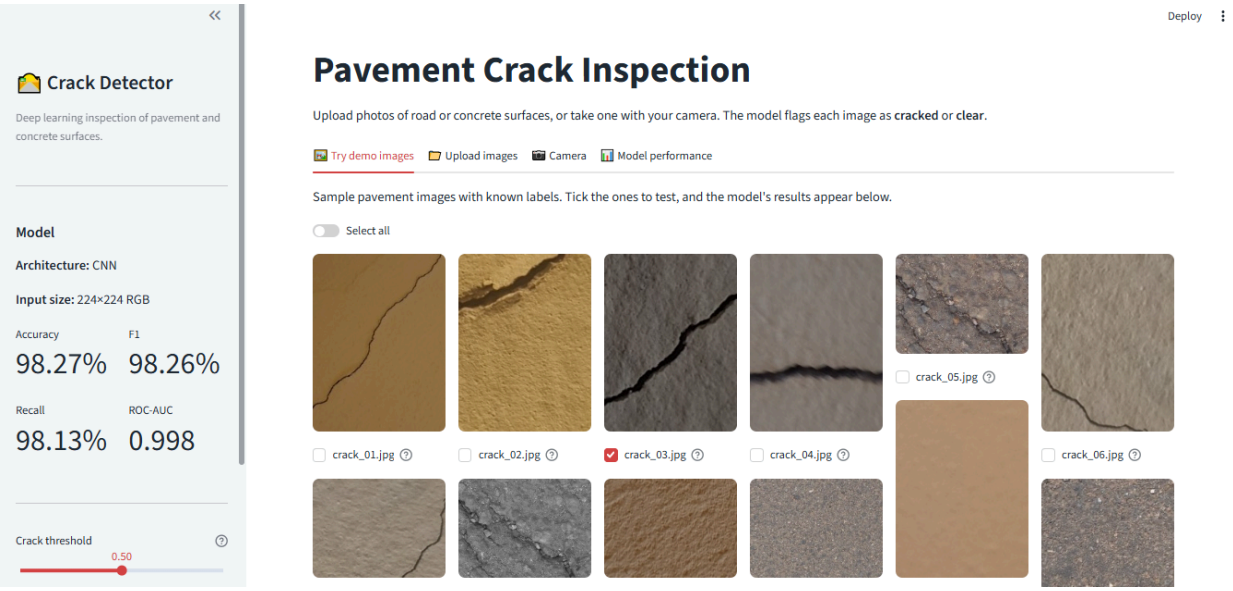


Figure 11: *Demo gallery: select sample images to test the model*

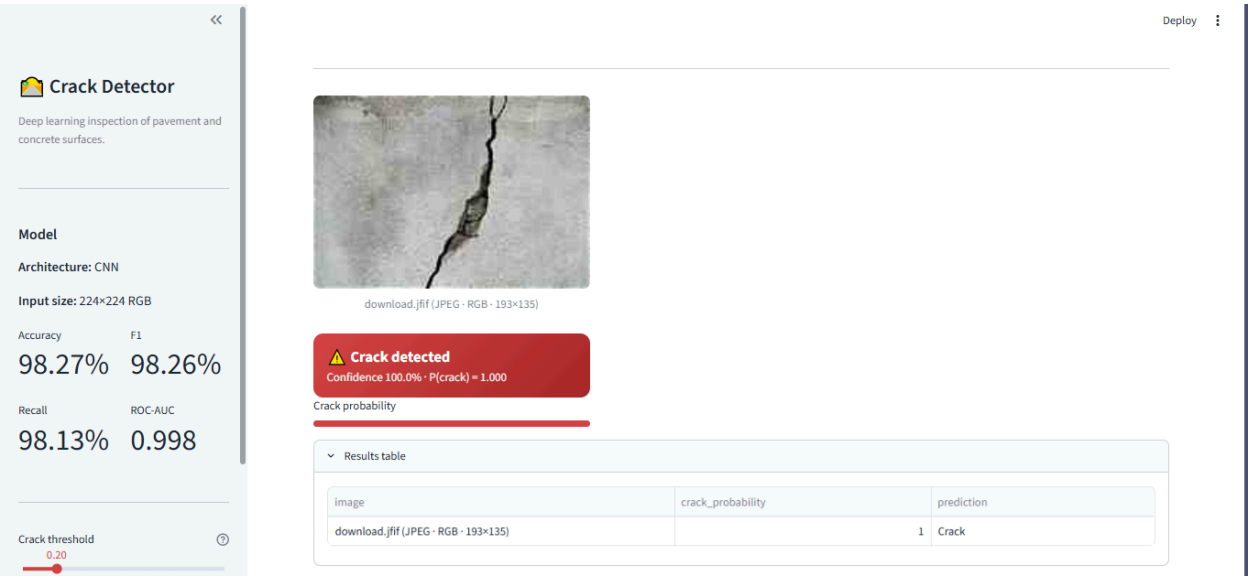


Figure 12: *Prediction on a web photo that is not from the dataset (.jfif, 193×135 px): crack detected with P(crack) = 1.000*

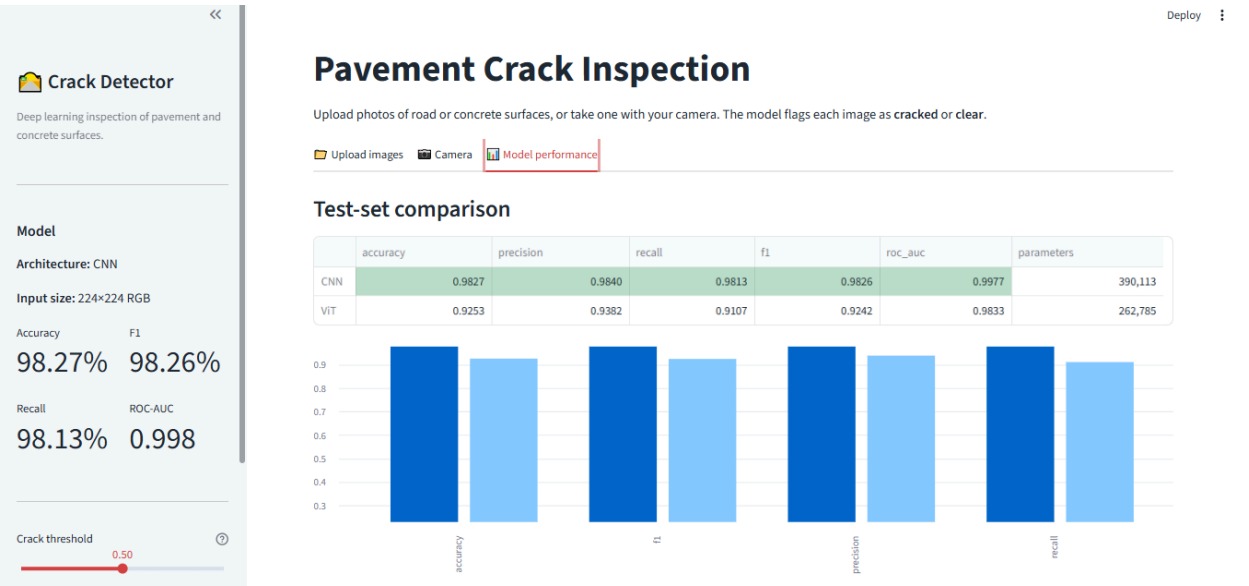


Figure 13: *Model performance tab: test-set comparison of the CNN and the ViT*

Run locally (Windows, inside the crack_detection_app folder):

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

Deployment: push the folder to GitHub and deploy it free on **Streamlit Community Cloud**, or use the included **Dockerfile** for Hugging Face Spaces, Render or any Docker host. **Tech stack:** TensorFlow / Keras 3, Streamlit, Pillow + pillow-heif, pandas, NumPy.

7. Conclusion and Future Work

A complete crack-detection system was built, from raw images to a deployed web app. With careful preprocessing, augmentation and modern training techniques (AdamW, learning-rate scheduling, early stopping), a compact CNN of only 390K parameters reached **98.27% accuracy, 0.9826 F1 and 0.9977 ROC-AUC** on 4,500 unseen test images, and clearly outperformed a ViT trained from scratch. Future work:

- Train for more epochs and try transfer learning (EfficientNet, a pre-trained ViT).
- Add **crack localization** (U-Net segmentation) and **severity classification** (slight, moderate, severe).
- Show **Grad-CAM** heatmaps in the app to explain which region triggered a prediction.
- Run the model on drone or mobile camera images for large-scale road inspection.

8. Project Links and Files

Google Colab notebook: pavement_crack_detection.ipynb

Deploy Project Link: [Pavement Crack Detector · Streamlit]